

exercise

Build your own GPT-4 Tokenizer!

Karpathy's original text from [minbpe/exercise.md](#) is quoted verbatim below. Everything under **Khmer** is my own, and marks where his assumptions change for a 3-byte script.

Step 1

Write the `BasicTokenizer` class, with the following three core functions:

- `def train(self, text, vocab_size, verbose=False)`
- `def encode(self, text)`
- `def decode(self, ids)`

Train your tokenizer on whatever text you like and visualize the merged tokens. Do they look reasonable? One default test you may wish to use is the text file `tests/taylorswift.txt`.

Khmer. Corpus is `khmer_train.txt` (25.0M chars) / `khmer_train_small.txt` (2.0M chars, use this one — pure Python BPE on the full corpus takes ~90 min). Hold out `khmer_test.txt` and never train on it.

"Do they look reasonable?" reads differently here. The first ~116 merges are **not** reasonable-looking and shouldn't be: Khmer codepoints are 3 UTF-8 bytes and every one of them starts `\xe1\x9e` or `\xe1\x9f`, so BPE spends its opening budget rebuilding the alphabet before it learns any Khmer. Merge 256 is `\xe1\x9e`, and the first whole character doesn't appear until merge 262.

Some merged tokens are also **not valid UTF-8** — they straddle a character boundary, e.g. `\x92\xe1\x9e`. That is normal, not a bug: `cl100k_base` itself has 773 such tokens. Print them with `errors="backslashreplace"`, and prefix `U+25CC` to any token starting with a combining mark or the terminal will stack it onto your separator.

Step 2

Convert your `BasicTokenizer` into a `RegexTokenizer`, which takes a regex pattern and splits the text exactly as GPT-4 would. Process the parts separately as before, then concatenate the results. Retrain your tokenizer and compare the results before and after. You should see that you will now have no tokens that go across categories (numbers, letters, punctuation, more than one whitespace). Use the GPT-4 pattern:

```
GPT4_SPLIT_PATTERN = r"""'(?i:[sdmt]|ll|ve|re)|[^\r\n\p{L}\p{N}]?
+\p{L}+|\p{N}{1,3}| ?[^\s\p{L}\p{N}][+|\r\n]*|\s*(?!S)|\s+"""
```

Khmer — his expected outcome does not hold. Measured at vocab 2048, held out:

pattern	chars/token
none (no pre-split)	2.786

Use the `regex` module, not `re`. minbpe's shipped `KHMER_SPLIT_PATTERN` uses Java/ICU syntax `(\x{17B6}, [a&&[^\b]])` and never compiles in Python.

both your encode and decode, you'll have to shuffle bytes around accordingly. If you're stuck, reference the `minbpe/gpt4.py` file for hints.

Khmer. Pure mechanics, nothing script-specific. Worth doing once for the byte-shuffle insight, but it teaches nothing about Khmer — skip it if time is short.

Step 4

(Optional, irritating, not obviously useful) Add the ability to handle special tokens. You'll then be able to match the output of `tiktoken` even when special tokens are present, e.g.:

```
import tiktoken
enc = tiktoken.get_encoding("cl100k_base") # this is the GPT-4 tokenizer
ids = enc.encode("<|endoftext|>hello world", allowed_special="all")
```

Without `allowed_special` `tiktoken` will error.

Khmer. One special token does matter eventually: U+200B (ZWSP) as a word-boundary marker, which is how PrahokBART and the Khmer OCR literature handle it. But ZWSP is unreliable as a *found* signal — density in this corpus is wildly uneven, and segmenting on it yields "words" averaging 25.6 characters, which are really clauses. It works as an annotation convention, not as naturally occurring evidence.

Step 5

If you've made it this far, you're now a pro at LLM Tokenization! Sadly, you're not exactly done yet because a lot of LLMs outside of OpenAI (e.g. Llama, Mistral) use `sentencepiece` instead. Primary difference being that `sentencepiece` runs BPE directly on Unicode code points instead of on UTF-8 encoded bytes. Feel free to explore `sentencepiece` on your own (good luck, it's not too pretty), and stretch goal if you really experience and suffer from the burden of time, re-write your BPE to be on Unicode code points and match the Llama 2 tokenizer.

Khmer — this is the step that actually matters, not the stretch goal.

For English, codepoints vs bytes is a curiosity: one character is one byte either way. For Khmer it is the whole game. Running BPE on codepoints skips the ~116-merge alphabet reconstruction entirely and eliminates boundary-straddling tokens by construction. That is why `SentencePiece` beats byte-level BPE here.

Two settings silently corrupt Khmer and must be set explicitly:

```
spm.SentencePieceTrainer.train(
    model_type='bpe',
    normalization_rule_name='identity', # default nmt_nfkc reorders
    Khmer marks
    split_by_whitespace=False,          # Khmer words are not
    whitespace-delimited
    byte_fallback=True,
```

```
character_coverage=1.0,  
)
```

Open question, apparently untested: KCC as the *base alphabet* — replacing the 256 bytes rather than constraining the merges. Pre-splitting on KCC is known to underperform (see Step 2), but nobody seems to have tried starting BPE from a KCC vocabulary with byte fallback. It would skip assembly, kill the straddling tokens, and leave merges unconstrained.

Progress

Step	Status	Note
1	done	<code>get_stats</code> / <code>merge</code> / <code>encode</code> / <code>decode</code> in <code>token.ipynb</code> , round-trip verified on held-out text
2	done, opposite conclusion	GPT-4 pattern costs 29% on Khmer; None wins
3	not started	mechanics only
4	not started	optional
5	next	the real research question

Reference numbers

Byte-level BPE trained on 25.0M chars, evaluated on 652,460 held-out chars:

tokenizer	vocab	chars/token
GPT-4 <code>cl100k_base</code>	100,277	0.619
mine	512	1.551
GPT-4o <code>o200k_base</code>	200,019	1.639
mine	2,048	2.666
mine	4,096	3.250
mine	16,384	4.498
mine	32,768	5.124

512 Khmer-specific tokens beat GPT-4's 100,277 by 2.5x. ~600 match GPT-4o's 200,019.

Caveats worth keeping attached to those numbers: the curve never plateaus, which is suspicious — test and train are both Wikipedia, and at 32k vocab over 25M chars many tokens appear a handful of times, so part of this is memorisation. And `cl100k/o200k` never saw this corpus while mine trained on its sibling. The defensible claim is *a small language-specific vocabulary vastly outperforms a large multilingual one on its own language*, not "beats GPT-4o".

Run `report(merges)` from `eval_tokenizer.py` for per-category breakdown.